

PV Energy Suite for Home Assistant

Forecast solar production, plan flexible consumption, manage negative prices and monitor phase load - a complete guide to PV Grid Forecast, Self-Consumption Planner, Phase Peak Guard and the PV switching blueprint.



What this suite is designed for

PV Energy Suite brings four separate functions together without unnecessarily tying them to one another. Only the PV blueprint may control the PV switch. Forecast, planner and Phase Guard analyse and advise.

Based on the current repository and README, rechecked on 5 October 2026. The README has actually changed since the earlier review; the new quick start and device configuration are incorporated in this guide.

1. The README has changed - and installation is clearer because of it

Yes: the README has been updated since my earlier review. The underlying architecture is unchanged, but the entry path is much clearer. The current README now opens with an explicit Start here section and recommends PV Grid Forecast with the integrated planner for a first test.

The most important clarifications are:

- work from your existing AppDaemon apps.yaml; do not overwrite that file;
- for the integrated planner, copy both `pv_grid_forecast.py` and `pv_self_consumption.py` and set `planner_enabled: true`;
- choose deliberately between fixed YAML values and Home Assistant helpers for device profiles;
- the README now explains much more explicitly which source entities you must provide yourself and which MQTT entities the suite creates automatically;
- the warning never to run the integrated and standalone planner at the same time is now more prominent and more concrete.

Important for this article

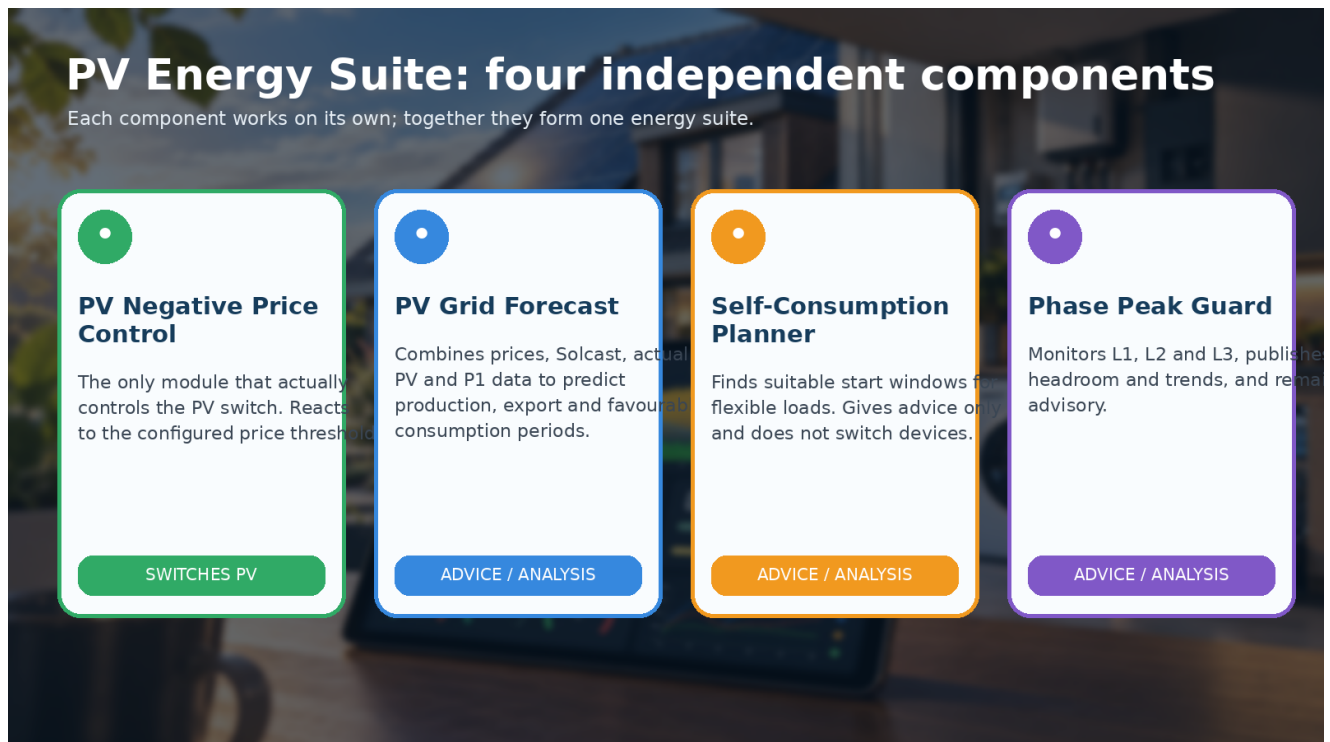
The installation chapters below follow this newer README structure, not the older version.

Contents of this guide

1. What the suite is · 2. The four components · 3. Data sources · 4. PV switching · 5. PV Grid Forecast · 6. Self-Consumption Planner · 7. Integrated or standalone · 8. Phase Peak Guard · 9. Installation · 10. Helpers and dashboards · 11. Example day · 12. Limitations, safety and maintenance.

2. One suite, but deliberately four independent components

PV Energy Suite is not one large monolithic automation. The repository bundles four components that can run independently. That matters: you can start small and expand later without needing the complete architecture from day one.



The division of responsibility is strict. The blueprint switches. The forecast predicts. The planner advises. Phase Guard monitors. That makes it clear which component is responsible when something happens.

Strong design principle

The PV blueprint is the only writer to the Shelly/PV switch. Forecast and planner read the same threshold and blocking helpers so their predictions are based on the same reality, but they do not control the hardware themselves.

3. What data goes into the suite?

The suite does not create source data itself. It combines existing Home Assistant entities from price, solar forecast, inverter and smart-meter integrations. The current README now emphasises this very clearly.

Source	Used by	Expected form
Zonneplan quarter-hour prices	Forecast + planner	forecast attribute with quarter-hour prices
Solcast today/tomorrow	Forecast + planner	detailedForecast attribute
Current PV power from ECU/inverter	Forecast + planner	W
P1 grid import and grid export	Forecast + planner	kW
Threshold + auto + manual block	Curtailement forecast + blueprint	Home Assistant helpers
Three phase sources	Phase Guard	preferably current in A

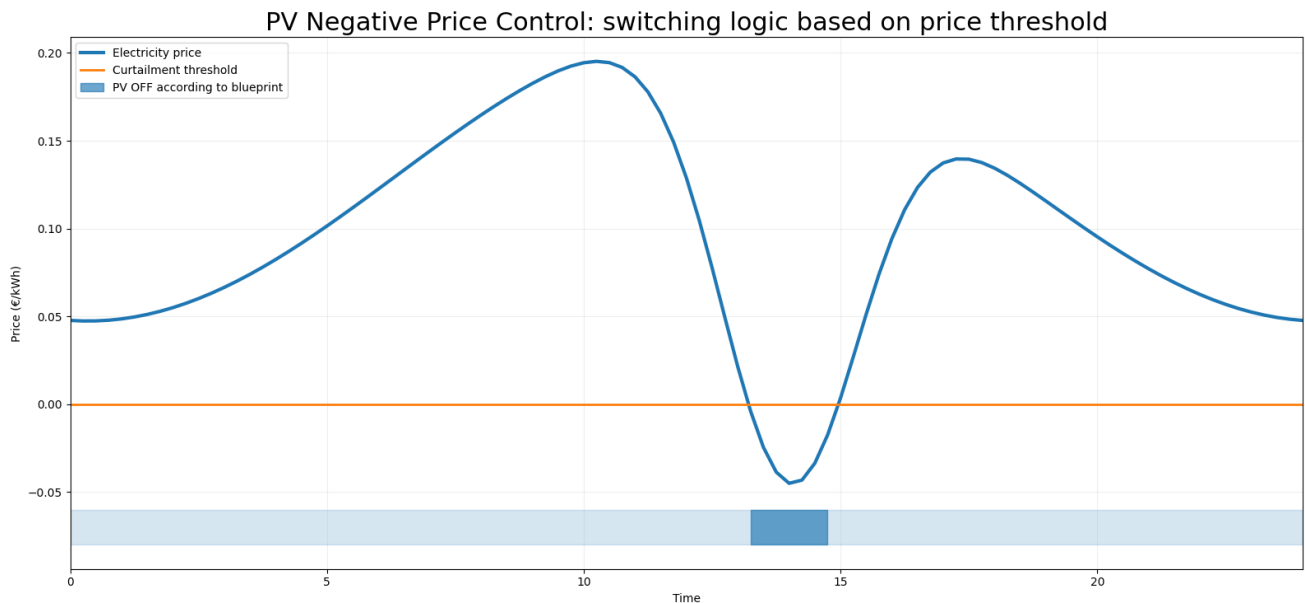
Units are not a minor detail. The repository expects ECU PV power in watts, P1 import/export in kilowatts and Solcast detailedForecast in kilowatts. Wrong units can otherwise produce results that look very convincing but are technically wrong.

MQTT Discovery

Forecast, planner and Phase Guard publish their results through MQTT Discovery. You therefore do not create sensors such as `sensor.pv_*`, `sensor.pv_planner_*` and `sensor.phase_guard_*` yourself as helpers.

4. PV Negative Price Control - the only component that actually switches

The blueprint compares the current electricity price with a configurable threshold. If the price is less than or equal to that threshold, the PV system can be switched off. When the price rises above the threshold again, PV can be switched back on.



The blueprint supports inverted switching logic. This matters when a contactor is wired so that switch on actually means PV is off. The README is therefore right to stress that you must always verify `inverted_switch` against the physical installation.

There are also helpers for automatic control, manual blocking, last switching time and minimum switching time. The blueprint additionally supports an optional temperature sensor.

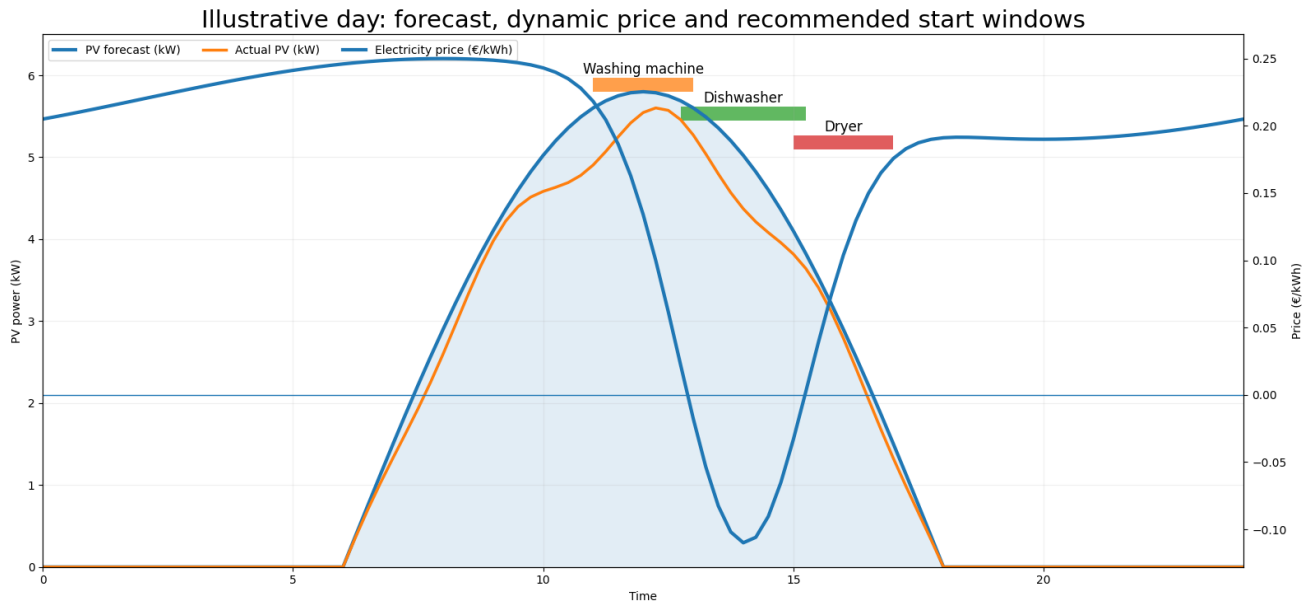
Also read the hardware article

For the electrical side - Shelly, contactor, fail-safe behaviour and physical wiring - this guide deliberately refers to the existing MechScript article: [Switching solar panels with a Shelly and contactor](#). This guide stays on the software side.

5. PV Grid Forecast - from separate sensors to one energy picture

PV Grid Forecast combines quarter-hour prices, Solcast, actual PV power and P1 import/export. The result is not merely a solar forecast, but a timeline on which price, expected production, actual behaviour and grid flow can be assessed together.

The app can also use learning and financial functions when the additional ECU and Zonneplan daily sensors are configured. Existing learning JSON files should be preserved during upgrades.

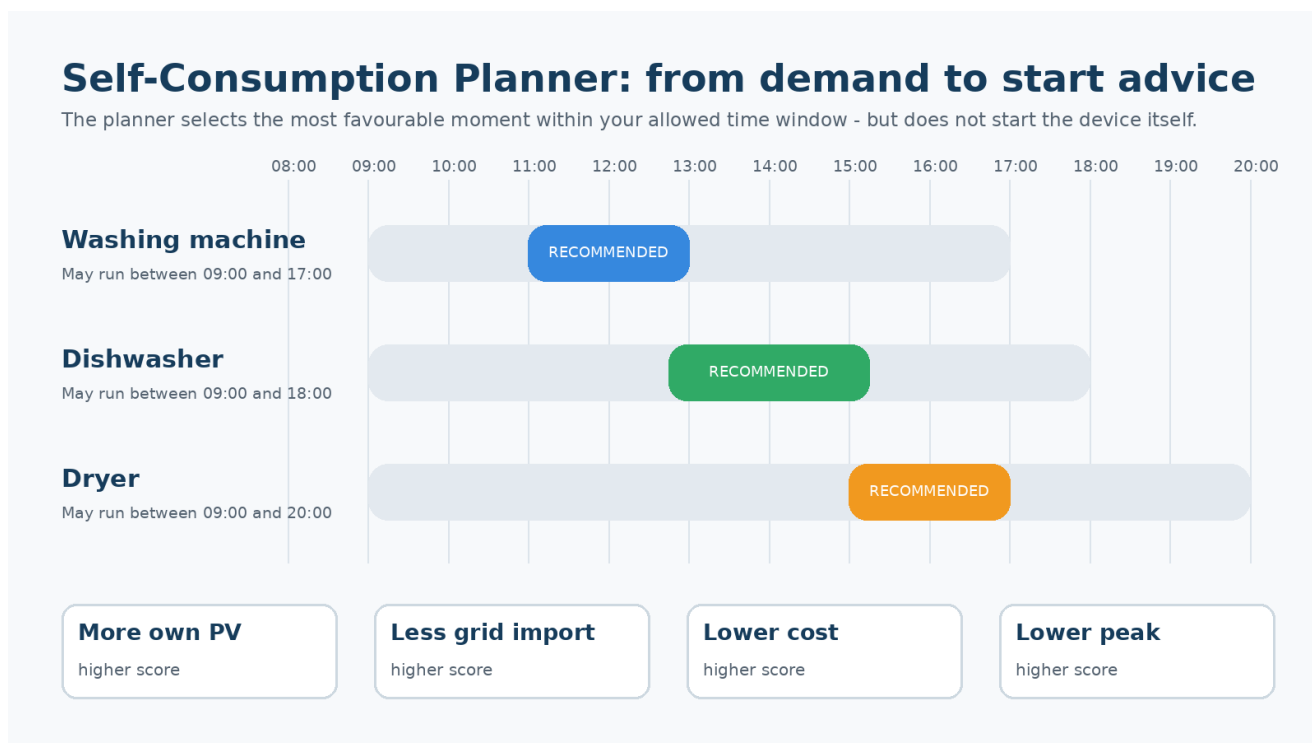


The graph above is illustrative, but it shows the idea behind the suite well: high solar production is not automatically the best moment. The planner also considers price, grid import and foregone export value.

PV Grid Forecast can also run the planner internally. According to the current README, that is the recommended route when you first want to test the suite.

6. Self-Consumption Planner - plan consumption intelligently without taking over devices

The Self-Consumption Planner is the most distinctive new component in the suite. Within an allowed time window, it looks for a favourable start time for a device. The standard examples are dishwasher, washing machine and dryer, but you can add additional devices.



For each device, the planner knows values such as programme duration, estimated average power, peak power, earliest start and latest finish. A device only participates in planning when there is actual demand.

Deliberately advisory-only

The planner does not start a washing machine, dryer or dishwasher. Not even when Phase Guard and the forecast are installed. It only provides advice. That is not a missing feature, but a deliberate safety and architecture choice.

7. How does the planner choose a good time window?

In the code, every candidate window is evaluated. The score rewards self-consumption and export reduction and penalises grid import, electricity costs and peak load. These factors have configurable weights.

More specifically, the calculation considers:

- how much of the programme consumption can come directly from expected PV production;
- how much export is avoided as a result;
- how much energy still needs to be imported from the grid;
- what that import costs and what export revenue is foregone by using the energy yourself;
- how large the estimated grid peak becomes during the programme.

The planner can also apply `pv_safety_margin_pct`. That prevents it from planning as if the solar forecast were guaranteed to be hit exactly.

The source code summarises the principle compactly: more self-consumption and export reduction increase the score; grid import, cost and peak load reduce it. So the planner clearly does more than simply “choose the sunniest hour”.

Important distinction

Global Phase Guard headroom is not used as a hard start safety check for a device whose phase is unknown. The repository explicitly warns about this.

8. Optional power sensor: the planner can learn from real cycles

A device profile can optionally include a current power sensor. The sensor must measure only that device and provide current power in W or kW. The planner samples the measurement once per minute.

After a completed cycle, the planner can use the measured consumption profile for future advice. In the code, average power is based on the median of stored cycle averages; estimated peak power is based on the highest observed peak in those cycles.

A single 0 W reading while the machine is off therefore does not suddenly become the predicted programme consumption. Programme duration remains deliberately manual because different programmes on the same appliance can differ substantially in duration.

The current README makes this distinction clear: measurement can make the power estimate more realistic, but the user remains responsible for the chosen programme duration and deadline.

Practical setting

If you measure 1.2 kWh over a 2-hour programme, 600 W is a reasonable first estimate of average power. Measure peak power separately when relevant. Then use the actual power sensor so the planner can refine its model.

9. Integrated planner or standalone? Choose exactly one

The suite supports two ways to run the planner. They publish the same MQTT Discovery IDs and topics. Therefore they must never be active at the same time.

Mode	Files	Configuration	Forecast basis
Forecast + planner	pv_grid_forecast.py + pv_self_consumption.py	pv_grid_forecast: with planner_enabled: true	can use PV Grid Forecast forecast/learning
Standalone planner	pv_self_consumption.py + pv_self_consumption_app.py	pv_self_consumption: from standalone_planner.yaml.example	reads sources itself; raw Solcast P50
Forecast only	pv_grid_forecast.py	planner_enabled: false	no planner output

If you are just starting

Use the integrated planner. That is now also the explicit quick start in the README. Only use standalone mode once you have a concrete reason for doing so.

10. Phase Peak Guard - making the electrical limit visible

Phase Peak Guard looks at the three phases separately. The app publishes current, headroom and trends through MQTT Discovery. It does not switch anything off; its output is intended for dashboards and human decision-making.

Phase Peak Guard: how much headroom does each phase have

Illustrative example for a 3×25 A connection. The app measures, analyses and advises - it switches nothing.



That matters on a 3×25 A connection: total power can look perfectly normal while one phase is much more heavily loaded than the other two.

Also read the dedicated article

The full operation, trend analysis and interpretation of Phase Peak Guard is already covered in the existing MechScript article: [Phase Peak Guard - Home Assistant](#). This suite guide therefore keeps that explanation deliberately compact.

11. Installation - the current quick start from the README

Installation in five verifiable steps

The README now explicitly says: work from your existing apps.yaml and choose one planner mode.



Files

Copy only the apps you actually use.



apps.yaml

Copy only the correct example block; do not overwrite your existing file.



Source entities

Replace REPLACE_ME and verify units and entity IDs.



AppDaemon + MQTT

Restart AppDaemon and verify logs and MQTT Discovery entities.



Dashboards

Add the optional Lovelace views only after the data is correct.

The recommended first test is Forecast + integrated planner. The steps are:

- copy apps/pv_grid_forecast.py and apps/pv_self_consumption.py to the directory that contains your AppDaemon apps.yaml;
- open your existing apps.yaml and copy only the pv_grid_forecast: block from the example;
- set planner_enabled: true and do not add a separate pv_self_consumption: block;
- replace all example entities and REPLACE_ME values you use with your own entity IDs;
- restart AppDaemon and check the log first, then the MQTT Discovery entities.

Do not overwrite

The current README rightly emphasises that apps.yaml.example is an example. Never overwrite your existing AppDaemon configuration with the example file.

12. The core of apps.yaml - what do you really need to understand?

The fragment below shows only the architecture. The actual repository contains more fields. Always use the current examples/apps.yaml.example as your source and substitute your own entity IDs.

Simplified example - do not copy blindly

```
pv_grid_forecast:
  module: pv_grid_forecast
  class: PVGridForecast
  planner_enabled: true
  price_quarter_entity: sensor.zonneplan...
  solcast_today_entity: sensor.solcast..._today
  solcast_tomorrow_entity: sensor.solcast..._tomorrow
  ecu_power_entity: sensor.your_pv_power
  grid_import_entity: sensor.power_consumption
  grid_export_entity: sensor.power_production
  threshold_entity: input_number.pv_negative_price_threshold
  automatic_entity: input_boolean.pv_automatic_curtailment
  manual_block_entity: input_boolean.pv_manual_block
  advisory_only: true
  automation_enabled: false
```

The key message is the division of roles: source entities come from Home Assistant; the app uses them; the results reappear as MQTT Discovery entities. In the public example, the planner is explicitly set to `advisory_only: true` and `automation_enabled: false`.

For Phase Guard, add a separate `phase_peak_guard`: block with three phase sources and its own JSON storage path.

13. Configuring devices: option A or option B

The updated README now separates this neatly into two routes.

Option A - quick test with fixed YAML values

In one device block, remove all fields ending in `_entity`. Then enter duration, average power, peak power, earliest start and latest finish directly in YAML. Set `enabled: true`. Note: with fixed values, the device stays in the advice every day until you set `enabled` back to `false`.

Option B - Home Assistant helpers and dashboard

Enable Home Assistant packages and place `planner_helpers.yaml.example` as `packages/pv_planner_helpers.yaml`. Each device gets six basic helpers plus an optional text helper for the power sensor.

Helper	Function
<code>input_boolean</code>	device should be planned
<code>input_number duration</code>	programme duration in minutes
<code>input_number average power</code>	initial estimate / manual value
<code>input_number peak power</code>	expected peak
<code>input_datetime earliest start</code>	start of allowed window
<code>input_datetime latest finish</code>	deadline
<code>input_text power sensor</code>	optional sensor.* entity for live measurement

14. The four included dashboard views

The repository contains four Lovelace views. They are individual views for the raw YAML editor, not complete dashboards. That distinction matters.

View	Purpose	Extra frontend cards
PV Grid Forecast	forecast, grid status, consumption advice, timeline	Mushroom, Layout Card, ApexCharts, Fold Entity Row, card-mod
PV overview	your own PV/P1/price/Shelly overview	Mushroom, Solar Forecast Card, Restriction Card, card-mod
Phase Guard	L1/L2/L3, headroom and trends	no extra cards
Device planning	helpers and planner advice per device	no extra cards

The forecast view mainly uses output created by the app itself via MQTT. The PV overview is different: many of its cards expect your own PV, P1, price and Shelly entities. Before using it, search for `replace_me` and `REPLACE_WITH_`.

Good order

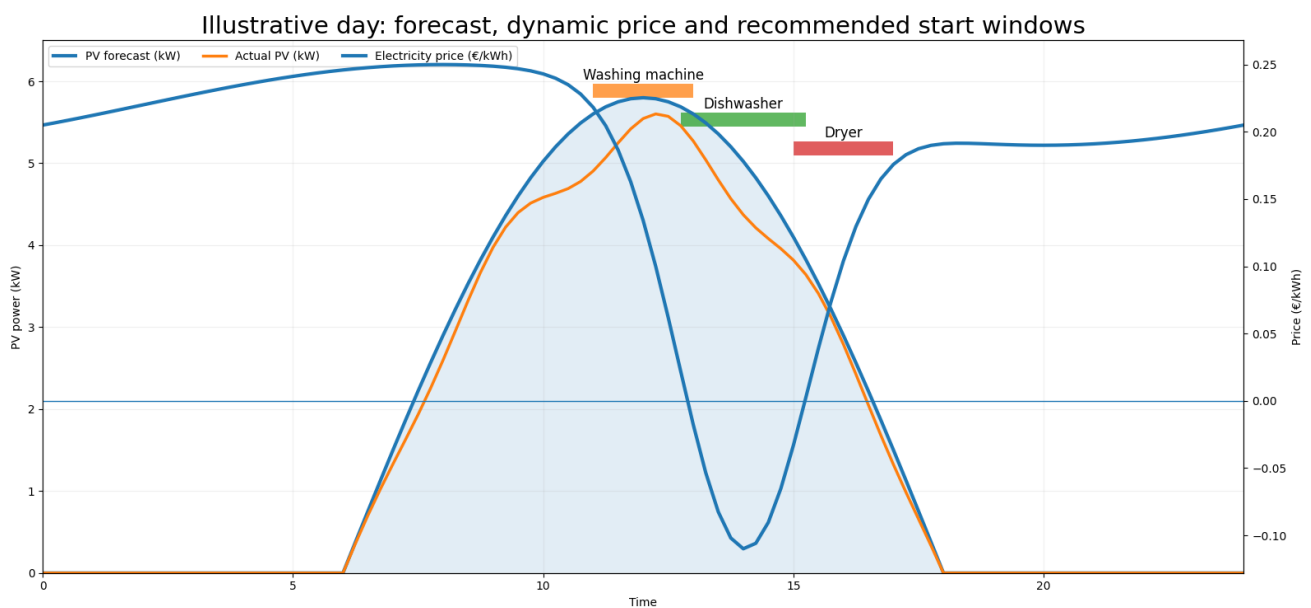
First make sure the apps and MQTT entities work correctly. Only then import dashboards. A beautiful dashboard cannot repair a wrong data model.

15. What does a complete day look like in practice?

Suppose Solcast predicts strong production around midday, while quarter-hour prices fall and part of the afternoon becomes negative. At the same time, you want the washing machine and dishwasher to run today.

The suite can then be used roughly like this:

- the forecast builds the timeline from price, Solcast, actual PV and P1 data;
- the planner evaluates candidate windows for each requested device within the configured deadlines;
- you might see, for example, that 11:00-13:00 is favourable for the washing machine and 12:45-15:15 for the dishwasher;
- the blueprint independently evaluates whether the current price falls below the configured PV threshold and can then operate the PV switch;
- Phase Guard simultaneously keeps following all three phases so the dashboard shows how much electrical headroom remains.



This example is illustrative. The suite predicts and advises using your real source data; it does not guarantee production or price and does not start household appliances itself.

16. What does PV Energy Suite deliberately not automate?

This may be the most important chapter for confidence in the system.

- The Self-Consumption Planner does not start devices.
- Phase Peak Guard does not switch consumers off.
- PV Grid Forecast does not operate the PV switch.
- Only the PV Negative Price Control blueprint may write to the PV Shelly/switch.
- The suite does not create your source entities; you need the correct integrations yourself.
- Global phase headroom is not a hard start safety check for a device on an unknown phase.

These boundaries are healthy. They prevent a forecast module from suddenly controlling hardware, or a planner without knowledge of the physical phase from treating a heavy load as “safe”.

For automatic device control

If you later want devices to start automatically, build that as a separate automation with explicit conditions and safeguards. Use planner advice as an input, not as implicit permission.

17. Commissioning, troubleshooting and upgrades

After installation, check the AppDaemon log first and only then the dashboards. Next, verify that the MQTT devices are online and that source entities provide the expected units.

Typical problems:

- `insufficient_data`: one or more required sources are missing or do not provide usable data; the planner then clears the current advice;
- `no planner entities`: integrated planner not active, MQTT not working correctly or AppDaemon has an error;
- `odd planning`: check duration, average power, peak power, time window and source units;
- `duplicate or unstable MQTT entities`: verify that integrated and standalone planner are not running at the same time;
- `PV switches the wrong way`: verify `inverted_switch` against the physical relay logic.

For upgrades, one rule is especially important: keep existing JSON learning files. Each standalone app should also have its own storage path; do not share JSON files between apps.

Sources and links

[GitHub - dkwolf1/pv-energy-suite](#)

[MechScript - Phase Peak Guard](#)

[MechScript - Switching solar panels with Shelly and contactor](#)

18. Conclusion

PV Energy Suite is particularly interesting because it does not throw different energy problems into one big automation. Forecasting, consumption planning, phase monitoring and PV curtailment each have their own responsibility. That lets you deploy the suite on a small scale and expand it in a controlled way.

The updated README makes the entry path noticeably better: first test Forecast + integrated planner, choose one configuration method for devices, explicitly verify source entities, and only then add dashboards and extra components. That is exactly the order I would recommend for a new installation.

In short

The suite turns separate energy data into a useful decision system: predict what is coming, advise when to consume, monitor what your electrical connection can handle, and automate switching only where that behaviour was explicitly designed.